IMPLEMENTATION MANUAL

Virtual Football · Score 2+ Probability Analyser

Premier League Historical Fixture Data Pipeline & Web-Based Probability Tool

---

Prepared by

Benjamin Fadina

Document Date: June 20, 2026

Version 1.0

Document Control

Field	Detail
Document Title	Implementation Manual — Virtual Football Score 2+ Probability Analyser
Author	Benjamin Fadina
Date Prepared	June 20, 2026
Version	1.0
Source Data File	PremierLeague_Results.xlsx
Delivered Application	index.html (single-file web application)
Classification	Project Documentation
GitHub Repository	https://github.com/benjaminsqlserver/VirtualFootballJune2026
Live Deployment	https://benjaminsqlserver.github.io/VirtualFootballJune2026/

Table of Contents

1. Introduction and Purpose

This manual documents the complete implementation of the Virtual Football Score 2+ Probability Analyser, a self-contained web application built to convert raw Premier League fixture history into an interactive decision-support tool. The work covers two deliverables that were studied and are described in full below:

- PremierLeague_Results.xlsx — the source workbook containing raw historical match results and a derived head-to-head probability table.
- index.html — a single-file, browser-based application that consumes the derived probability table and lets a user build a slip of up to ten fixtures, then instantly view each team's historical likelihood of scoring two or more goals in that specific head-to-head matchup.

The purpose of the tool is to give a user a fast, visual way to assess goal-scoring tendencies between specific team pairings, using only data that already exists in the historical fixture record — no external APIs, servers, or databases are required at runtime.

The complete project, including the source workbook and the application file, is version-controlled and published in the following GitHub repository: https://github.com/benjaminsqlserver/VirtualFootballJune2026.

A live, browser-ready version of the application is also deployed and publicly accessible at https://benjaminsqlserver.github.io/VirtualFootballJune2026/.

2. Source Data: PremierLeague_Results.xlsx

The workbook contains two worksheets. The first holds the raw match log; the second holds a pre-aggregated probability table derived from it.

2.1 Worksheet: Master

The Master sheet is the raw fixture ledger. It contains 1,520 data rows (plus a header row) across six columns:

Column	Description	Example
FixtureID	Sequential unique identifier for each recorded match	1
HomeTeam	Three-letter code for the home side	MNU
HomeScore	Goals scored by the home team	1
AwayTeam	Three-letter code for the away side	LIV
AwayScore	Goals scored by the away team	0
Season	Season index the fixture belongs to (1–4 observed)	1

Twenty teams are represented using three-letter codes, each playing every other team home and away across four recorded seasons — for example ARS (Arsenal), MNC (Man City), MNU

(Man United), LIV (Liverpool), CHE (Chelsea), TOT (Tottenham), and 14 others. This produces a full round-robin history of 380 fixtures per season, i.e. 1,520 rows across the four seasons captured in the file.

2.2 Worksheet: Score2+Probability

The second sheet, Score2+Probability, is a derived summary table with 380 rows (one per unique home/away team pairing) and four columns:

Column	Description
HomeTeam	Three-letter code of the designated home side for this pairing
AwayTeam	Three-letter code of the designated away side for this pairing
HomeTeam 2+ Goals Probability	Fraction of the four recorded head-to-head meetings (one per season) in which the home team scored 2 or more goals, e.g. "3/4"
AwayTeam 2+ Goals Probability	Fraction of the same four meetings in which the away team scored 2 or more goals, e.g. "1/4"

Each pairing's denominator is 4 because the Master sheet contains exactly four seasons of fixtures for every home/away combination of the twenty teams. The numerator counts, for that exact home/away orientation, how many of the four matches saw the respective side reach two or more goals.

2.3 Derivation Logic (Master → Score2+Probability)

The probability sheet is a one-to-one aggregation of the Master sheet, computed as follows:

1. Group all Master rows by the (HomeTeam, AwayTeam) pair, which yields 380 groups of 4 rows each (one row per season).
2. Within each group, count how many rows have HomeScore ≥ 2; this count becomes the numerator of "HomeTeam 2+ Goals Probability", expressed as a fraction over 4.
3. Within the same group, count how many rows have AwayScore ≥ 2; this becomes the numerator of "AwayTeam 2+ Goals Probability", also expressed as a fraction over 4.
4. Write one summary row per (HomeTeam, AwayTeam) pair into the Score2+Probability sheet, preserving the fractional notation (e.g. "2/4") rather than a decimal, so the underlying sample size remains visible to the user at a glance.

This pre-aggregation means the web application never needs to scan the full 1,520-row match log at runtime — it works directly from the 380-row summary, which keeps the page lightweight and the lookup instantaneous.

3. Application Overview: index.html

index.html is a single, self-contained HTML file — branded "VF · Score 2+ Probability Analyser" — that embeds the entire Score2+Probability dataset as an inline JavaScript array. There is no backend, no build step, and no external data fetch: the file can be opened directly in any modern browser and is fully functional offline.

3.1 Visual Design

- A dark pitch-green theme (deep green background, bright accent green highlights, amber/red status colours) reinforcing the football subject matter.
- Typography pairs 'Barlow Condensed' for headings/labels (a tall, sporty display face) with 'Inter' for body text and data, loaded from Google Fonts.
- A circular badge logo and a two-line header ("VIRTUAL FOOTBALL · SCORE 2+ ANALYSER" / "Premier League · Historical Probability Data") establish the tool's identity.
- The layout is responsive: a media query at 640px collapses the fixture-row grid and hides the in-table probability bar track on small screens, keeping the page usable on mobile.

3.2 Functional Sections

The interface is organised into two stacked panels:

5. Step 1 — Select Fixtures: a fixture builder offering ten rows, each with a Home Team and an Away Team dropdown, a live counter ("X / 10 added"), an Analyse Fixtures button, a Clear All button, and a per-row clear control.
6. Results: a sortable table that appears once fixtures are analysed, showing each fixture's home team, away team, both teams' 2+ goal probabilities (as fraction, percentage, and a colour-tiered horizontal bar), and a combined score.

4. In-Browser Data Model

On load, the script builds three in-memory structures from the embedded dataset:

Structure	Role
RAW	The full array of 380 objects copied verbatim from the Score2+Probability sheet ({HomeTeam, AwayTeam, HomeTeam 2+ Goals Probability, AwayTeam 2+ Goals Probability})
LOOKUP	A hash map keyed by "HOME_AWAY" (e.g. "ARS_BOU") built from RAW, giving O(1) retrieval of a pairing's probability record
TEAMS / TEAM_NAMES	TEAMS is the sorted, de-duplicated list of three-letter codes derived from RAW; TEAM_NAMES is a hand-maintained dictionary mapping each code to its full club name (e.g. MNC → "Man City") for display purposes

Two small helper functions support the model:

- parseFraction(s) — converts a string like "3/4" into a decimal (0.75) for sorting, bar-width, and colour-tier calculations.
- tierClass(val) — buckets a decimal probability into one of five visual tiers (tier-0, tier-25, tier-50, tier-75, tier-100), each mapped to a distinct bar colour.

5. Core Workflows

5.1 Building the Fixture Slip

buildFixtureUI() dynamically generates ten identical fixture rows. Each row contains a Home Team <select> and an Away Team <select>, both populated from TEAMS with the full club name and code shown together (e.g. "Man City (MNC)"). A delegated change listener on the container calls updateCounter() and updateAnalyseBtn() whenever any dropdown changes, and a delegated click listener handles the per-row clear (×) button.

5.2 Validating and Counting Selections

getSelections() reads all ten rows and returns only the rows where both a home and an away team have been chosen, tagged with their original fixture number. updateCounter() reflects the count in the "X / 10 added" label, and updateAnalyseBtn() enables the Analyse Fixtures button only once at least one complete fixture exists.

5.3 Running the Analysis

Clicking Analyse Fixtures triggers the following sequence for every completed row:

7. Reject identical home/away selections (logged as "same team" and surfaced via a toast notification).
8. Build the lookup key "HOME_AWAY" and retrieve the matching record from LOOKUP.
9. If no record exists for that exact orientation, the fixture is added to a notFound list and skipped.
10. If a record is found, parse both fraction strings into decimals, and compute combined = homeProb + awayProb.
11. Collect all successfully resolved fixtures into originalRows and currentRows, reset sorting to fixture order, render the results table, reveal the Results panel, and smooth-scroll it into view.

Any unresolved fixtures are reported together in a single toast message rather than silently dropped, so the user always knows whether every row of their slip was matched.

5.4 Sorting and Resetting

The results toolbar offers six sort buttons (#, Home Team, Away Team, Home 2+ %, Away 2+ %, Combined). Clicking a button toggles direction if it is already the active sort column, or switches column (defaulting to descending for the three probability-based columns and ascending for the others). sortAndRender() performs a stable comparison — string comparison for team names, numeric comparison for probabilities — and re-renders the table without re-querying the underlying dataset. A Reset Order button restores currentRows to the original analysed order (originalRows) and resets the sort state.

5.5 Rendering Results

renderTable() writes one table row per analysed fixture, each showing: the fixture number; the home team name with a "HOME" chip; a probability bar combining the raw fraction, a colour-tiered fill bar (width proportional to probability) and a rounded percentage; the equivalent away

team cell with an "AWAY" chip; and a combined-probability percentage coloured by combinedColor() (accent green ≥1.5, amber ≥1.0, light grey ≥0.5, muted grey below 0.5). If no fixtures resolve to data, an empty-state message is shown instead of an empty table.

6. Supporting UX Elements

- Toast notifications: showToast(msg, isError) renders a temporary bottom-right notification (3-second auto-dismiss) used for both confirmations (e.g. "All fixtures cleared", "Grid reset to original order") and warnings (unmatched fixtures).
- Clear All: instantly blanks every dropdown across all ten rows, recalculates the counter and button state, and hides the results panel.
- Empty/error states: a dedicated #emptyState block communicates clearly when a selected combination simply does not exist in the dataset, rather than showing a blank table.

7. Technical Summary

Aspect	Implementation
Delivery format	Single static HTML file; no server, build tooling, or external runtime dependency beyond a Google Fonts stylesheet import
Data source	Embedded JSON array (380 records) generated from the Score2+Probability worksheet of PremierLeague_Results.xlsx
Lookup strategy	In-memory hash map keyed by "HOMECODE_AWAYCODE" for constant-time retrieval
Maximum slip size	10 fixtures per analysis run (configurable via the MAX constant)
Sorting	Client-side array sort on six selectable columns, ascending/descending toggle
Styling	Hand-rolled CSS using custom properties (CSS variables) for a consistent dark pitch-green theme; responsive breakpoint at 640px
Browser compatibility	Any evergreen browser supporting ES6 (template literals, arrow functions, Set, template-based DOM construction)

8. How to Use the Tool

12. Open index.html in any modern web browser — no installation or internet connection is required after the first load (aside from the Google Fonts request) — or simply visit the hosted version at https://benjaminsqlserver.github.io/VirtualFootballJune2026/.
13. In the Select Fixtures panel, choose a Home Team and an Away Team for each fixture you want to analyse, up to ten.

14. Click Analyse Fixtures. The Results panel appears showing each fixture's historical 2+ goals probability for both sides.
15. Use the sort buttons to reorder by team name or by probability strength, or click Reset Order to return to the original input order.
16. Click Clear All at any point to start a fresh slip.

9. Maintenance and Extension Notes

- To refresh the probabilities for a new season, recompute the Score2+Probability sheet from an updated Master sheet using the derivation logic in Section 2.3, then regenerate the embedded RAW array in index.html from the updated sheet.
- To add a new team, ensure it has fixture rows in Master for every existing team (home and away), regenerate Score2+Probability, add its three-letter code and full name to TEAM_NAMES, and refresh RAW.
- The MAX constant (currently 10) controls the number of fixture rows offered in the builder and can be adjusted without any other code changes.
- Because RAW is embedded directly in the HTML file, no database or API integration is required; refreshing data is purely a matter of replacing the RAW array with newly exported JSON.
- Both project files, along with their version history, are maintained in the GitHub repository at https://github.com/benjaminsqlserver/VirtualFootballJune2026; future updates to either file should be committed and pushed there to keep the repository in sync with the live deployment at https://benjaminsqlserver.github.io/VirtualFootballJune2026/.

— End of Manual —

Prepared and authored by Benjamin Fadina.